

End-to-End Incremental Data Pipeline using Databricks (Medallion Architecture)

1. Project Overview

This project implements an end-to-end batch data pipeline using Databricks and Delta Lake following the Medallion Architecture pattern.

The pipeline processes NYC Yellow Taxi trip data through Bronze, Silver, and Gold layers while implementing incremental processing, partition optimization, and workflow orchestration.

The project demonstrates practical data engineering concepts including:

- Data ingestion
- Data transformation
- Incremental processing
- Delta Lake operations
- Merge-based upserts
- Partitioning strategies
- Workflow orchestration
- Business-level aggregations
- Data quality handling

2. Technologies Used

Technology	Purpose
Databricks	Data engineering platform
PySpark	Distributed data processing
Delta Lake	ACID storage layer
Databricks Workflows	Pipeline orchestration
Unity Catalog Volumes	Storage management
SQL / Spark SQL	Data querying
Python	Transformation logic

3. Dataset Information

Dataset Used: NYC Yellow Taxi Trip Data

Files:

- yellow_tripdata_2026_01
- yellow_tripdata_2026_02

Key Columns:

Column	Description
tpep_pickup_datetime	Pickup timestamp
tpep_dropoff_datetime	Dropoff timestamp
passenger_count	Passenger count
trip_distance	Distance traveled
fare_amount	Fare amount
total_amount	Total trip amount
VendorID	Taxi vendor identifier

4. Architecture Design

Medallion Architecture

The project follows a layered Medallion Architecture:

```
Raw Data
  ↓
Bronze Layer
  ↓
Silver Layer
  ↓
Incremental Merge
  ↓
Gold Layer
```

5. Storage Structure

Volume Structure

```
/Volumes/workspace/default/project_vol/  
├── bronze/  
│   └── taxi_data  
├── silver/  
│   └── taxi_data  
└── gold/  
    ├── daily_revenue  
    ├── trips_per_hour  
    └── fare_efficiency
```

6. Pipeline Layers

6.1 Bronze Layer

Purpose

The Bronze layer stores raw ingested data with minimal transformation.

Responsibilities:

- Raw ingestion
- Preserve original schema
- Store source data as-is
- Serve as landing layer

Bronze Logic

Input

Source Table:

```
spark.table("workspace.default.yellow_tripdata_2026_01")
```

Output

```
/Volumes/workspace/default/project_vol/bronze/taxi_data
```

Write Strategy

```
.mode("overwrite")
```

Bronze Notebook

```
bronze_df = spark.table("workspace.default.yellow_tripdata_2026_01")

bronze_df.write.format("delta")
    .mode("overwrite")
    .save("/Volumes/workspace/default/project_vol/bronze/taxi_data")
```

6.2 Silver Layer

Purpose

The Silver layer contains cleaned, validated, and transformed data.

Responsibilities:

- Data cleaning
- Schema standardization
- Derived columns
- Validation handling
- Partitioning

Cleaning Logic

Invalid records removed:

```
(trip_distance == 0)
AND
(fare_amount == 0)
AND
(passenger_count == 0)
```

Derived Columns

pickup_date

Derived from:

```
tpep_pickup_datetime
```

Used for:

- Partitioning
- Aggregation
- Query optimization

Partition Strategy

Partitioned by:

```
pickup_date
```

Benefits:

- Query pruning
- Faster reads
- Reduced scan cost
- Better organization

Silver Notebook

```
from pyspark.sql.functions import col, to_date

silver_df = spark.read.format("delta")
    .load("/Volumes/workspace/default/project_vol/bronze/taxi_data")

clean_df = silver_df.filter(
    ~(
        (col("trip_distance") == 0) &
        (col("fare_amount") == 0) &
        (col("passenger_count") == 0)
    )
)

clean_df = clean_df.withColumn(
    "pickup_date",
```

```
        to_date(col("tpep_pickup_datetime"))
    )

clean_df.write.format("delta")
    .mode("overwrite")
    .partitionBy("pickup_date")
    .save("/Volumes/workspace/default/project_vol/silver/taxi_data")
```

6.3 Incremental Processing Layer

Purpose

The incremental layer processes newly arriving data without reprocessing the entire dataset.

Responsibilities:

- Process new data
- Avoid duplicates
- Maintain idempotency
- Merge new records safely

Why Incremental Processing?

A full refresh pipeline:

- Reprocesses all data
- Increases compute cost
- Slows execution
- Is inefficient at scale

Incremental processing:

- Processes only new data
 - Reduces cost
 - Improves scalability
 - Supports production workloads
-

Incremental Source

```
workspace.default.yellow_tripdata_2026_02
```

Merge Strategy

Instead of append-only writes, Delta MERGE was used.

Benefits:

- Prevents duplicates
- Supports reruns safely
- Maintains consistency
- Enables idempotent processing

Merge Keys

```
VendorID  
+  
tpep_pickup_datetime
```

Incremental Notebook

```
from pyspark.sql.functions import col, to_date  
from delta.tables import DeltaTable  
  
new_df = spark.table("workspace.default.yellow_tripdata_2026_02")  
  
new_df_clean = new_df.withColumn(  
    "pickup_date",  
    to_date(col("tpep_pickup_datetime"))  
)  
  
silver_table = DeltaTable.forPath(  
    spark,  
    "/Volumes/workspace/default/project_vol/silver/taxi_data"  
)  
  
silver_table.alias("target").merge(  
    new_df_clean,  
    "target.pickup_date = new_df_clean.pickup_date",  
    "update_or_insert"
```

```
new_df_clean.alias("source"),  
"""  
target.tpep_pickup_datetime = source.tpep_pickup_datetime  
AND target.VendorID = source.VendorID  
"""  
).whenNotMatchedInsertAll().execute()
```

6.4 Gold Layer

Purpose

The Gold layer stores curated business-level datasets for analytics and reporting.

Responsibilities:

- Business aggregations
- KPI generation
- Reporting datasets
- Analytical outputs

Gold Datasets

1. Daily Revenue

Objective

Calculate total revenue generated per day.

Logic

```
groupBy("pickup_date")
```

Output

```
/gold/daily_revenue
```

2. Trips Per Hour

Objective

Analyze taxi demand trends by hour.

Logic

```
hour(tpep_pickup_datetime)
```

Output

```
/gold/trips_per_hour
```

3. Fare Efficiency

Objective

Calculate average fare earned per kilometer.

Special Handling

Handled divide-by-zero scenarios safely.

Output

```
/gold/fare_efficiency
```

Gold Notebook

```
from pyspark.sql.functions import sum, avg, hour, when, col

silver_df = spark.read.format("delta")
    .load("/Volumes/workspace/default/project_vol/silver/taxi_data")

# Daily Revenue

daily_revenue = silver_df.groupBy("pickup_date")
    .agg(sum("total_amount").alias("total_revenue"))
```

```

# Trips Per Hour

df_hour = silver_df.withColumn(
    "pickup_hour",
    hour("tpep_pickup_datetime")
)

trips_per_hour = df_hour.groupBy("pickup_hour")
    .count()

# Fare Efficiency

efficiency = silver_df.withColumn(
    "fare_per_km",
    when(col("trip_distance") > 0,
        col("fare_amount") / col("trip_distance"))
).groupBy("pickup_date")
    .agg(avg("fare_per_km").alias("avg_fare_per_km"))

base_path = "/Volumes/workspace/default/project_vol/gold"

# Writes

daily_revenue.write.format("delta")
    .mode("overwrite")
    .save(f"{base_path}/daily_revenue")

trips_per_hour.write.format("delta")
    .mode("overwrite")
    .save(f"{base_path}/trips_per_hour")

efficiency.write.format("delta")
    .mode("overwrite")
    .save(f"{base_path}/fare_efficiency")

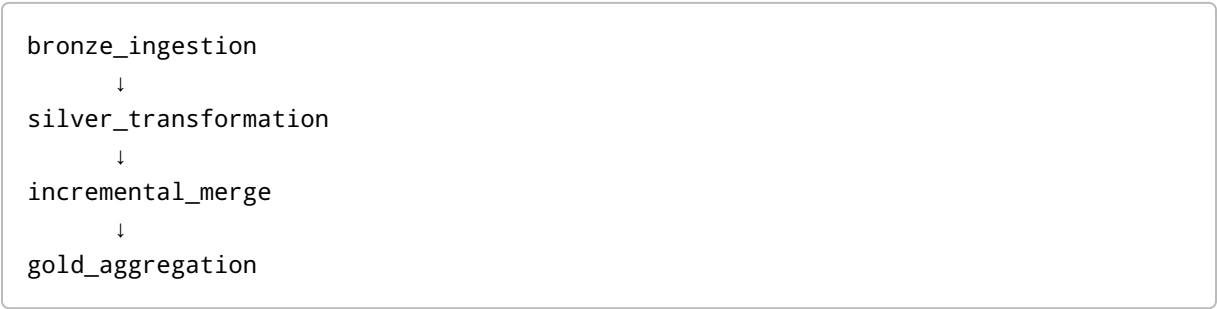
```

7. Workflow Orchestration

Tool Used

Databricks Workflows

Workflow DAG



Workflow Benefits

- Automated execution
- Dependency management
- Retry handling
- Sequential orchestration
- Pipeline monitoring

Tasks Created

Task Name	Responsibility
bronze_ingestion	Raw data ingestion
silver_transformation	Cleaning and partitioning
incremental_merge	Merge new data
gold_aggregation	Generate business metrics

8. Data Engineering Concepts Implemented

Concept	Implemented
Medallion Architecture	Yes
Delta Lake	Yes
Incremental Processing	Yes
Merge Upserts	Yes

Concept	Implemented
Partitioning	Yes
Workflow Orchestration	Yes
Data Validation	Yes
Derived Columns	Yes
Business Aggregations	Yes

9. Challenges Faced and Resolutions

Challenge 1

Issue

Partition column not found.

Root Cause

pickup_date column did not exist before partitioning.

Resolution

Derived pickup_date using:

```
to_date(tpcp_pickup_datetime)
```

Challenge 2

Issue

Delta metadata mismatch.

Root Cause

Existing Silver table schema did not match incremental schema.

Resolution

Recreated Silver table with proper partitioning and schema.

Challenge 3

Issue

Divide-by-zero error in Gold layer.

Root Cause

Some records had trip_distance = 0.

Resolution

Used conditional handling:

```
when(col("trip_distance") > 0)
```

Challenge 4

Issue

DBFS root disabled.

Root Cause

Workspace restricted public DBFS usage.

Resolution

Used Unity Catalog Volumes instead of /FileStore.

10. Performance Considerations

Sampling Strategy

A sampled dataset was used during development due to limited compute resources.

Reason:

- Faster iteration
- Reduced execution time

- Efficient testing in development environment

Important: Pipeline logic remains scalable to full dataset execution.

Partitioning Optimization

Silver layer partitioned by:

pickup_date

Benefits:

- Reduced data scans
 - Faster query execution
 - Improved performance
-

11. Production Improvements (Future Enhancements)

Potential enhancements:

1. Streaming Ingestion

Use:

- Auto Loader
 - Structured Streaming
 - Kafka
-

2. Cloud Storage Integration

Move storage to:

- Amazon S3
 - Azure Data Lake Storage Gen2
-

3. Incremental Gold Refresh

Instead of full Gold overwrite:

- Update only affected partitions
-

4. Data Quality Framework

Implement:

- Great Expectations
 - Deequ
 - Validation framework
-

5. Monitoring and Alerting

Add:

- Pipeline alerts
 - Logging
 - Failure notifications
-

12. Resume Description

Resume Bullet Points

- Built an end-to-end data pipeline using Databricks implementing Bronze, Silver, and Gold layers with Delta Lake
 - Designed incremental data processing using merge-based upserts to handle duplicate and newly arriving data
 - Optimized storage and querying using partitioning strategies and derived date columns
 - Orchestrated pipeline execution using Databricks Workflows with task dependencies and automated execution flow
 - Developed business-level analytical datasets including revenue trends, trip demand patterns, and fare efficiency metrics
-

13. Key Learnings

Key learnings from this project:

- Understanding Medallion Architecture
 - Designing layered data pipelines
 - Handling incremental processing
 - Using Delta MERGE for idempotent updates
 - Implementing partitioning strategies
 - Building orchestrated workflows
 - Managing schema-related issues
 - Understanding practical data engineering lifecycle
-

14. Conclusion

This project demonstrates an end-to-end modern data engineering pipeline using Databricks and Delta Lake.

The implementation covers:

- ingestion
- transformation
- incremental processing
- aggregation
- orchestration

while applying practical data engineering concepts used in real-world analytics pipelines.

The project provides hands-on experience with scalable data architecture patterns and workflow-driven processing.